



Indian Institute of Technology Guwahati

BSc. (Hons.) Data Science & Artificial Intelligence
Online Degree Programme

INTERNSHIP/TERM PROJECT

Forecasting Daily Equity Volatility: Can Machine Learning Beat Classical Models?

Student Name :	François Schmitt
Student ID :	23035010523
Email ID :	f.schmitt@op.iitg.ac.in
Programme :	BSc. (Hons.) Data Science & AI
Course Code :	DA 379
Course Name :	Term Project
Trimester :	IX

Introduction

What is the project about, and why?

Volatility: the size of an asset's daily price swings. It drives option pricing, value-at-risk, margins and position sizing.

A key asymmetry: the direction of tomorrow's return is unpredictable, but its size is strongly persistent, so it can be forecast.

Classical models: GARCH-type models have exploited this persistence for four decades.

The question: can machine learning (XGBoost) and deep learning (LSTM) forecast next-day volatility better than these classical baselines?

The approach: test every model fairly, in one harness, with no look-ahead bias, and check whether any difference is statistically real.

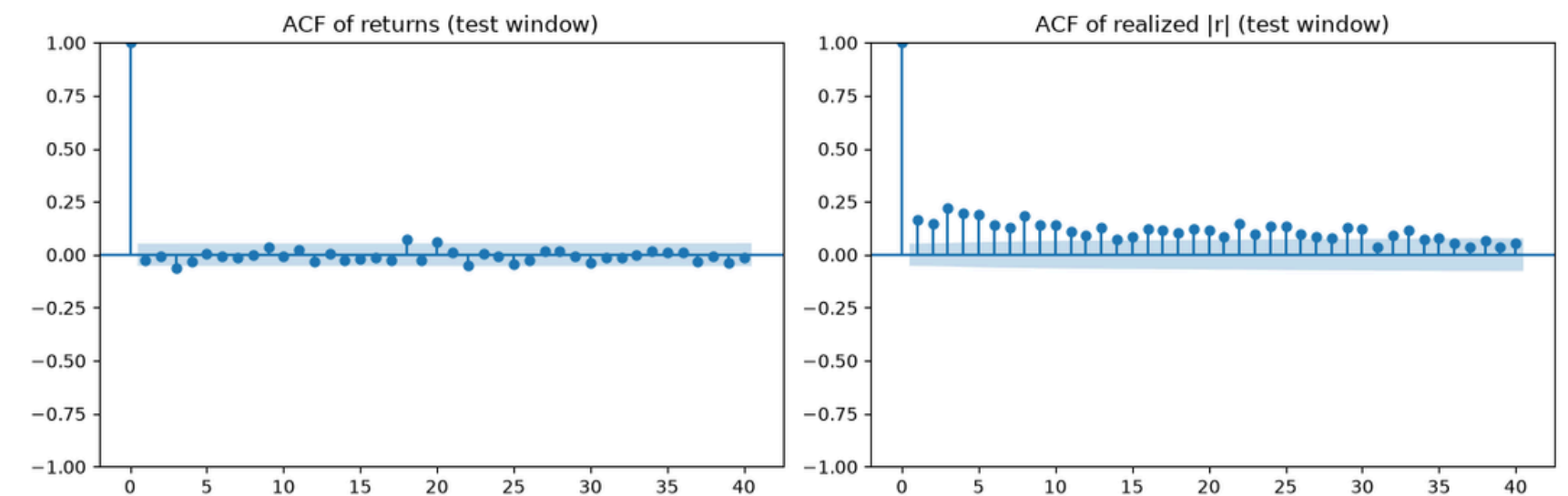
Why Volatility Can Be Forecast

Direction is noise; size is persistent

Returns: show no autocorrelation. Past returns do not predict the direction of the next one.

Absolute returns: show strong, slowly decaying autocorrelation. Big days cluster together.

Volatility clustering: this persistence is the one signal every model in this study exploits.



Objectives

1. Build honest classical baselines: naive, rolling, EWMA, and the GARCH family (GARCH, GJR-GARCH, EGARCH).
2. Build a leak-free walk-forward evaluation with RMSE, MAE, QLIKE and the Diebold–Mariano test.
3. Build a walk-forward XGBoost forecaster with justified features.
4. Build and regularize an LSTM forecaster with explicit anti-leakage guards.
5. Compare all models on one shared test window and test whether the differences are significant.
6. Deliver a reproducible, modular codebase.

Dataset: S&P 500 Daily Returns

Source: Yahoo Finance (^GSPC), auto-adjusted close

4,650

trading days

2008–2026

18.5 years

70 / 30

train / test split

1,395

out-of-sample days

Returns: daily log returns, the standard object for volatility models.

Preprocessing: log-difference adjusted closes; no other cleaning needed.

Test window: the final 30% (2020-12-07 to 2026-06-29), shared by every model.

Stress test: the window contains the 2022 bear market and the April-2025 tariff shock.

Tools & Technologies

Language

Python 3.13

Data

pandas
NumPy
yfinance

Models

arch (GARCH)
XGBoost
PyTorch (LSTM)
statsmodels

Workflow

Jupyter
VS Code
Git / GitHub
LaTeX

The five heavy model fits run in parallel processes; the full pipeline reproduces end-to-end in about one minute.

The Models

Eight forecasters, from three-parameter recursions to a neural network

Classical

Naive: yesterday's squared return

Rolling(21): variance of last 21 days

EWMA(0.94): RiskMetrics decay

GARCH(1,1): $\omega + \alpha r^2 + \beta \sigma^2$

GJR / EGARCH: add the leverage effect

Learned

XGBoost: gradient-boosted trees, refit monthly

LSTM: reads 20 days of features

Shared features: recent returns, realized variance

Shared target: log smoothed forward variance

Guarded: train-only scaling, early stopping

Evaluation: A Leak-Free Walk-Forward

No look-ahead: the forecast for a day uses only data up to the day before. All models share one test window.

Periodic refit: classical and ML models are re-estimated about monthly, then rolled forward on observed data.

QLIKE (headline): robust to noise in the squared-return proxy, and it punishes under-prediction of variance.

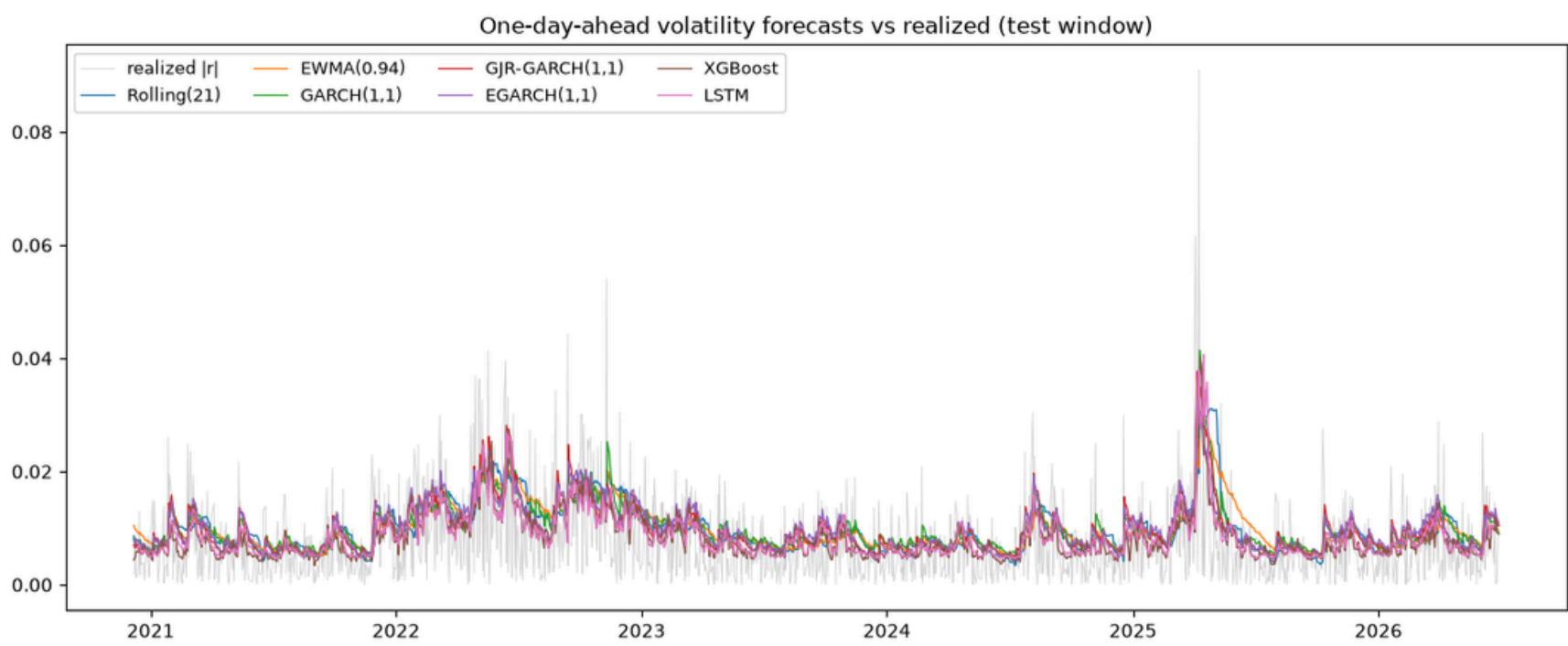
RMSE / MAE: point accuracy on the volatility scale, for interpretability.

Diebold–Mariano: tests whether a gap in loss between two models is statistically real, not luck.

Results: Leaderboard

Out-of-sample, 1,395 days, sorted by QLIKE (lower = better)

Model	RMSE	MAE	QLIKE
EGARCH(1,1)	0.00728	0.00556	1.5042
GJR-GARCH(1,1)	0.00731	0.00554	1.5091
LSTM	0.00686	0.00499	1.5287
GARCH(1,1)	0.00751	0.00576	1.5569
EWMA(0.94)	0.00750	0.00573	1.5814
XGBoost	0.00691	0.00496	1.5911
Rolling(21)	0.00762	0.00571	1.6344
Naive	0.00951	0.00672	5440.5



All models track the same clusters. EGARCH(1,1) wins on QLIKE; the LSTM ranks 3rd, ahead of plain GARCH; the learned models take the best RMSE (LSTM) and MAE (XGBoost).

Is the Winner Really Better?

The best model vs the best baseline

EGARCH(1,1) gives the lowest QLIKE; GJR-GARCH is statistically tied ($p = 0.47$).

Leverage matters: both beat plain GARCH(1,1) decisively ($p < 0.0001$) on this window.

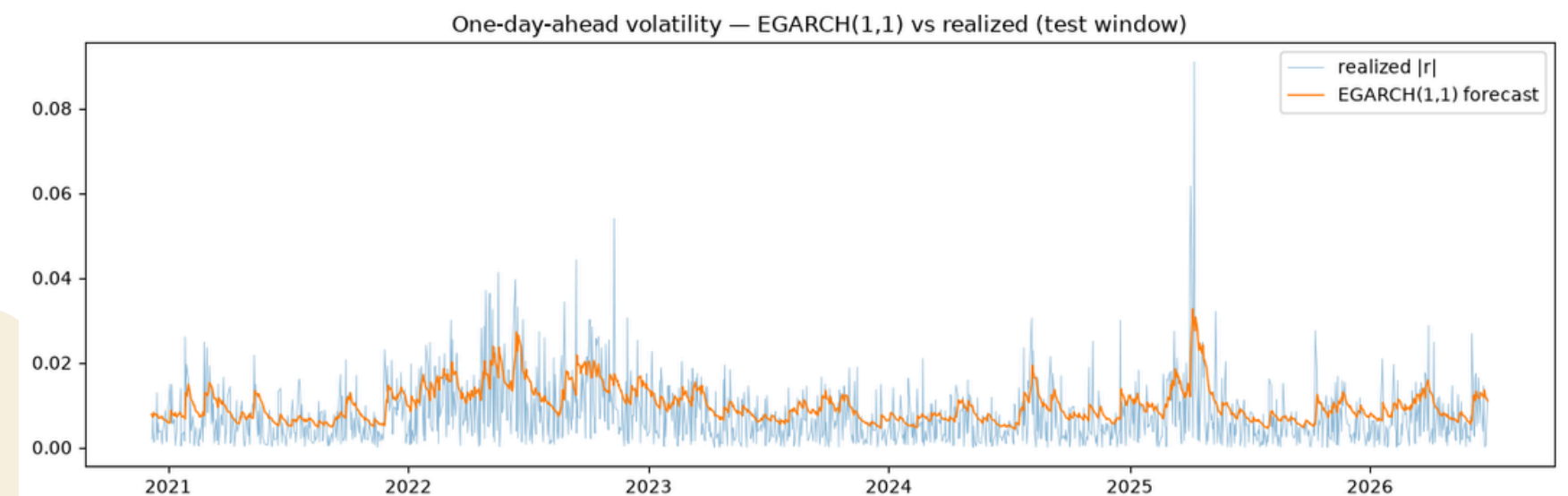
LSTM: third place, ahead of plain GARCH, and statistically tied with the winner ($p = 0.19$).

QLIKE 1.504

EGARCH(1,1), best model

$p < 0.0001$

EGARCH vs plain GARCH: significant



Why Do the Leverage Models Win?

Leverage effect: the test window is dominated by two sharp crashes (2022 and April 2025), and the leverage term turns a big fall directly into higher next-day variance.

Right objective: the GARCH family is fit by maximum likelihood on variance; the learned models train on a smoother substitute target.

Small data: roughly 3,250 noisy training days favours simple recursions over neural networks.

QLIKE asymmetry: it rewards fast reactions to spikes, and the GARCH recursion reacts the very next day.

Limitations: single asset and window, a noisy proxy, and deliberately modest hyperparameter tuning.

Conclusion & Takeaways

Answer: no. A monthly-refit EGARCH(1,1) gives the best QLIKE; no learned model beats the leverage GARCH family where it matters.

Nuance: the LSTM ranks third, ahead of plain GARCH, and is statistically tied with the winner; LSTM and XGBoost take the best RMSE and MAE.

Leverage matters: on a window with two asymmetric crashes, EGARCH and GJR significantly beat plain GARCH.

Main lesson: a leak-free walk-forward harness and the choice of metric matter more than model complexity.

Future work: intraday realized variance, extra inputs like the VIX, and multi-asset training.

Project artifacts

Code: github.com/iitgF/T9

Report: LaTeX (Overleaf)

Notebook: consolidated, runs end-to-end